Junior React js developer *Developing new user-facing features using React.js *Building reusable components and front-end libraries for future use *Translating designs and wireframes into high-quality code *Optimizing components for maximum performance across a vast array of web-capable devices and browsers| Skills Strong proficiency in JavaScript, including DOM manipulation and the JavaScript object model § Thorough understanding of React.js and its core principles § Experience with popular React.js workflows (such as Flux or Redux) § Familiarity with newer specifications of EcmaScript § Experience with data structure libraries (e.g., Immutable.js) § Knowledge of isomorphic React is a plus § Familiarity with RESTful APIs § Knowledge of modern authorization mechanisms,such as JSON Web Token § Familiarity with modern front-end build pipelines and tools § Experience with common front-end development tools such as Babel, Webpack, NPM,etc. § Ability to understand business requirements and translate them into Technical requirements § A knack for benchmarking and optimization § Familiarity with code versioning tools i have interview today help me to practise with the questions(with fully explained answers) likely asked from these topics as much as possible

Great Shubham — since you're already working on **MERN projects with JWT auth, React context, protected routes, and dashboards**, this interview is totally aligned with what you've been building. 💪

I'll simulate a **real Junior React Developer interview** with:

- 🔹 Technical questions
- 🔹 Scenario-based questions
- 🔹 Practical coding questions
- 🔹 Performance & optimization
- 🔹 Architecture & tools
- 🔹 JWT & REST questions

Each answer will be **fully explained** so you understand the *why*, not just the definition.

🔥 1. Core React Questions

1️⃣ What is React?

React is a **JavaScript library for building user interfaces**, especially single-page applications.

It is:

- Component-based
- Declarative
- Uses a Virtual DOM
- Unidirectional data flow

Created by: Facebook (now Meta)

Why React is powerful?

- Reusable components
- Efficient updates using Virtual DOM
- Easy state management
- Large ecosystem

2️⃣ What is Virtual DOM?

The **Virtual DOM** is a lightweight JavaScript representation of the real DOM.

How it works:

1. State changes
2. React creates a new Virtual DOM
3. Compares with previous Virtual DOM (Diffing)
4. Updates only changed parts in real DOM

This process is called **Reconciliation**.

Why important?

Direct DOM manipulation is slow.
React updates only what is necessary → better performance.

3️⃣ What are Components in React?

Components are reusable building blocks of UI.

Types:

1. Functional Components (Modern way)

```
JavaScriptfunctionWelcome() {
return<h1>Hello</h1>;
}
```

2. Class Components (Old way)

```
JavaScriptclassWelcomeextendsReact.Component {
  render() {
return<h1>Hello</h1>;
  }
}
```

👉 Today, functional components + hooks are preferred.

4️⃣ What is JSX?

JSX = JavaScript XML

It allows writing HTML inside JavaScript.

```
JavaScriptconstelement=<h1>Hello</h1>;
```

Behind the scenes, it becomes:

```
JavaScriptReact.createElement("h1", null, "Hello");
```

5️⃣ What is State?

State is a built-in object that stores data that may change over time.

JavaScriptconst [count, setCount] =useState(0);

When state changes → component re-renders.

6️⃣ What are Props?

Props = properties passed from parent to child.

JavaScriptfunctionChild({ name }) {
return<h1>{name}</h1>;
}

Props are:

- Read-only
- Immutable

🔥 2. Hooks (Very Important)

7️⃣ What is useState?

Allows functional components to have state.

JavaScriptconst [count, setCount] =useState(0);

- count → current value
- setCount → function to update

8️⃣ What is useEffect?

Used for:

- API calls
- Side effects
- Subscriptions

- Timers

```
JavaScriptuseEffect(() => {
fetchData();
}, []);
```

Dependency Array:

- [] → run once (componentDidMount)
- [value] → run when value changes
- no array → runs every render

9️⃣ Difference between useEffect and useLayoutEffect?

- useEffect → runs after paint
- useLayoutEffect → runs before paint

Use case: DOM measurement

1️⃣0️⃣ What is useContext?

Used for global state without prop drilling.

Example: AuthContext (you've already implemented this 🔥)

🔥 3. Redux / Flux

1️⃣1️⃣ What is Redux?

Redux is a **state management library**.

It has:

- Store
- Actions

- Reducers
- Dispatch

Flow:
Component → Dispatch → Reducer → Store → UI update

Why Redux?

- Predictable state
- Centralized state
- Good for large apps

1️⃣2️⃣ What is Reducer?

A pure function that takes:

- previous state
- action

Returns:

- new state

```
JavaScriptfunctioncounterReducer(state, action) {
switch(action.type) {
case"INCREMENT":
return { count: state.count +1 };
default:
returnstate;
  }
}
```

🔥 4. JavaScript Questions

1️⃣3️⃣ Explain Closures

A closure is when a function remembers variables from its outer scope.

```
JavaScriptfunctionouter() {
letcount=0;
returnfunctioninner() {
count++;
console.log(count);
  }
}
```

Used in:

- React hooks
- Event handlers

1️⃣4️⃣ What is Event Delegation?

Instead of adding event listeners to multiple elements,
we attach it to parent and use bubbling.

Improves performance.

1️⃣5️⃣ Difference Between let, var, const?

Feature	var	let	const
Scope	Function	Block	Block
Reassign	Yes	Yes	No
Hoisting	Yes	Yes	Yes

🔥 5. REST & JWT (VERY IMPORTANT FOR YOU)

1️⃣6️⃣ What is REST API?

REST = Representational State Transfer

Uses HTTP methods:

- GET → fetch
- POST → create
- PUT → update
- DELETE → delete

Stateless communication.

1️⃣7️⃣ What is JWT?

JWT = JSON Web Token

Used for authentication.

Structure:
Header.Payload.Signature

Flow:

1. User logs in
2. Server generates token
3. Client stores token (localStorage or httpOnly cookie)
4. Client sends token in Authorization header

CodeAuthorization: Bearer <token>

1️⃣8️⃣ Why JWT is secure?

- Signed
- Can be verified
- No session storage needed

🔥 6. Performance Optimization (Very Important)

1️⃣9️⃣ How to optimize React app?

1. React.memo()

Prevents unnecessary re-renders.

2. useMemo()

Memoizes computed values.

3. useCallback()

Memoizes functions.

4. Code Splitting

JavaScriptconstComponent=React.lazy(() => import('./Component'));

5. Lazy Loading

2️⃣0️⃣ What is Re-rendering?

Component re-renders when:

- State changes
- Props change
- Parent re-renders

Avoid unnecessary renders using:

- React.memo
- Pure components

🔥 7. Build Tools (Webpack, Babel, NPM)

2️⃣1️⃣ What is Babel?

Babel converts modern JavaScript (ES6+) to older JS.

2️⃣2️⃣ What is Webpack?

Module bundler.
Bundles JS, CSS, images into optimized files.

2️⃣3️⃣ What is NPM?

Node Package Manager.
Manages dependencies.

🔥 8. Scenario-Based Questions

2️⃣4️⃣ How would you protect routes in React?

Use:

- JWT token
- ProtectedRoute component

JavaScriptif (!token) return<Navigateto="/login"/>;

(You've already done this in your MERN project 🔥)

2️⃣5️⃣ How would you handle API errors globally?

- Axios interceptors

- Error boundary
- Toast notifications

2️⃣6️⃣ How do you structure a scalable React app?

Example structure (like yours):

```
Codesrc/
  components/
  pages/
  context/
  api/
  hooks/
  utils/
```

Keep:

- Reusable components separate
- Business logic separate
- API layer separate

🔥 9. Advanced Questions They May Ask

2️⃣7️⃣ What is Reconciliation?

Process of comparing old Virtual DOM with new one and updating efficiently.

2️⃣8️⃣ What are Keys in React?

Keys help React identify which items changed.

```
JavaScript{items.map(item => (
<likey={item.id}>{item.name}</li>
))}
```

Bad:

JavaScriptkey={index}

2️⃣9️⃣ What is Immutable Data?

Data that cannot be modified directly.

Instead of:

JavaScriptstate.push()

Use:

JavaScriptsetState([...state, newItem])

🔥 10. HR + Practical Questions

3️⃣0️⃣ Explain one project you built.

Since you built:

- MERN E-waste platform
- JWT auth
- Pickup request system
- Role-based access

You can say:

> I built a full-stack MERN application with JWT-based authentication, role-based access control, and RESTful APIs. On frontend, I implemented protected routes, Axios interceptors, reusable components, and dashboard-based UI architecture.

That answer will impress them.

🚀 Rapid Fire Questions (Answer in 1 Line)

- What is SPA? → Single Page Application
- What is prop drilling? → Passing props deeply through components
- What is lifting state up? → Moving state to common ancestor
- What is controlled component? → Input controlled by React state
- What is uncontrolled component? → Input controlled by DOM

🎯 Final Interview Strategy for You

Since you:

- Know MERN
- Implemented JWT
- Used Axios interceptors
- Built dashboards

👉 Highlight:

- Real-world experience
- Role-based auth
- API integration
- Performance optimization
- Folder structure

If you want, I can now:

- 🔥 Do a **mock live interview simulation**
- 🔥 Give coding questions to solve
- 🔥 Give tricky React debugging questions

- 🔥 Give HR + behavioral answers
- 🔥 Give 30 ultra-advanced React questions

Tell me how much time you have before interview ⏳